USDG GOLD (USDG)

SMART CONTRACT SECURITY SELF-ASSESSMENT REPORT

Internal / Source-Code Self-Assessment

Contract Name	USDGGold
Token Name	USDG Gold
Token Symbol	USDG
Token Standard	ERC-20 compatible
Solidity Version	0.8.20
Network	Ethereum-compatible EVM network
Assessment Date	13 August 2026
Assessment Status	SELF-ASSESSMENT — NOT AN INDEPENDENT AUDIT

IMPORTANT NOTICE
This document is an internal technical self-assessment based solely on the supplied Solidity source code. It is not an independent third-party audit, formal verification, certification, or guarantee of security.

Document purpose: present the supplied self-assessment in a professional audit-style format suitable for public disclosure, while clearly distinguishing it from an independent security audit.

1. Executive Summary

The USDGGold smart contract implements an ERC-20-compatible token with fixed initial total supply, owner-controlled administration, ERC-20 transfers and allowances, Uniswap V2/V3 pool address restrictions, one-way transfer-constraint removal, ownership transfer and renouncement, ERC-20 recovery functionality, metadata URI storage, and ETH receiving/recovery capability.

Overall Assessment

No obvious Critical or High-severity exploitable vulnerability was identified from the supplied source code. However, the contract contains meaningful centralization, operational, and design risks that should be disclosed to token users.

The most important operational finding is the owner-controlled transfer restriction involving the configured Uniswap V2 and V3 pool addresses. While **transferConstraints** remains enabled, transfers involving either configured pool are deliberately reverted. Normal DEX trading through those addresses therefore will not function until the owner executes **removeTransferConstraints()**.

Assessment methodology: source-code review of access control, arithmetic safety, reentrancy exposure, token-transfer logic, external calls, ownership management, supply management, and denial-of-service/design conditions.

2. Scope and Contract Architecture

Component	Description
Token accounting	totalSupply and balanceOf mappings
Allowances	approve and transferFrom
Ownership	owner, transferOwnership, renounceOwnership
Supply	Fixed at deployment; no mint mechanism identified
Transfer restriction	Controlled by transferConstraints
DEX configuration	Configured Uniswap V2 and V3 pool addresses
Recovery	ERC-20 and ETH recovery functions
Metadata	Immutable metadata URI after deployment through supplied interface
ETH handling	receive() and fallback() accept ETH

The supplied source is a single, non-proxy Solidity contract. No upgradeable proxy mechanism was identified in the reviewed source.

3. Token Supply Analysis

The constructor sets **maxSupply = maxSupply_**, initializes **totalSupply = maxSupply_**, and assigns the entire initial supply to the deployer. No public or owner-only mint function and no burn function were identified.

Assessment: Informational / Design Characteristic. The deployed supply is effectively fixed because the supplied contract contains no mechanism that increases totalSupply. The deployer receives 100% of the initial supply at deployment, which is a distribution and centralization consideration rather than a code vulnerability.

4. Access-Control Analysis

The contract uses an **onlyOwner** modifier requiring **msg.sender == owner**. Owner privileges include removing transfer constraints, transferring or renouncing ownership, recovering unrelated ERC-20 tokens, and recovering ETH.

AS-01 — Owner Centralization MEDIUM

The owner possesses significant administrative authority, particularly over the one-way removal of transfer restrictions and recovery of assets held by the contract. No unauthorized access path to these functions was identified from the supplied source.

Recommendation: For production deployment, consider multisig ownership, timelocked administrative actions, and explicit public documentation of owner privileges.

5. Transfer Restriction Analysis

The constructor initializes **transferConstraints = true**. When enabled, transfers involving either configured Uniswap V2 or V3 pool address revert.

AS-02 — DEX Transfers Blocked While Constraints Are Enabled HIGH (OPERATIONAL)

This is a high-severity operational/configuration risk rather than a conventional code-execution vulnerability. While the restriction is enabled, buying from or selling to the configured pools can fail, as can liquidity operations involving those addresses. The owner can call **removeTransferConstraints()**, after which the restriction cannot be re-enabled through the supplied interface.

Recommendation: Before live trading, verify both pool addresses, execute the removal only when intended, perform a small test transfer and swap, and publicly disclose the initial restricted state and administrative transition.

6. Reentrancy Analysis

Token transfer functions update balances directly and do not perform external calls. Recovery functions do perform external calls, but they are protected by **onlyOwner** and no obvious exploitable reentrancy path was identified from the supplied source.

Assessment: Low Risk.

7. Arithmetic Safety

The contract targets Solidity 0.8.20, which provides checked arithmetic by default. Balance and allowance subtraction is additionally guarded by explicit sufficiency checks. No obvious arithmetic overflow/underflow vulnerability was identified.

Assessment: Low Risk.

8. ERC-20 Transfer and Allowance Logic

The reviewed interface includes totalSupply, balanceOf, transfer, allowance, approve, and transferFrom, with standard Transfer and Approval events. The core accounting appears internally consistent.

The allowance model follows a conventional ERC-20 pattern. As with many ERC-20 implementations, replacing a non-zero allowance directly with another non-zero allowance can create the standard allowance race consideration in certain front-running scenarios. This is not unique to USDGGold.

9. Ownership Management

AS-03 — Single-Step Ownership Transfer LOW

Ownership is transferred immediately to the supplied new owner address. A mistaken transfer could therefore assign administrative control to an unintended address. A two-step ownership pattern and/or multisig is recommended for higher-security deployments.

Renouncement warning: If ownership is renounced while transferConstraints remains enabled, owner-only configuration actions—including removal of those constraints—can no longer be executed.

Ownership should not be renounced until all required deployment and trading configuration steps are complete.

10. Recovery and External Calls

AS-04 — Owner Can Recover Unrelated ERC-20 Tokens LOW / INFORMATIONAL

The owner may recover ERC-20 tokens held by the contract, while the supplied code prevents recovery of the USDGGold token itself. The owner may also recover ETH received by the contract. These are administrative custody functions protected by onlyOwner.

Assessment: No obvious unauthorized withdrawal path or exploitable reentrancy path was identified in the supplied source.

11. Metadata URI

The metadata URI is stored during construction and the supplied contract does not expose an owner function to change it. It is therefore effectively immutable through the reviewed interface after deployment.

12. Uniswap Pool Configuration

AS-05 — Immutable Pool Configuration INFORMATIONAL

The configured V2/V3 pool addresses have no setter in the supplied source. An incorrect deployment parameter cannot be corrected through the contract's administrative interface and may cause restrictions to apply to unintended addresses or fail to apply to intended pools.

13. Supply, Burn, Upgradeability and DoS Review

Supply / minting: No mint, mintTo, _mint, or other supply-expansion mechanism was identified. The owner cannot increase USDG supply through the supplied contract.

Burn: No burn function was identified. This is a design characteristic, not a vulnerability.

Upgradeability: No proxy, delegatecall upgrade path, implementation address, or upgrade function was identified. The supplied contract is not upgradeable through its own interface.

Denial of service: The pool-transfer reversion is an explicit design mechanism controlled by transferConstraints, not an arbitrary-user DoS exploit.

14. Findings Summary

ID	Finding	Severity	Classification
AS-01	Owner has significant administrative control	Medium	Design risk
AS-02	Uniswap pool transfers blocked while constraints enabled	High*	Operational risk
AS-03	Single-step ownership transfer	Low	Recommendation
AS-04	Owner can recover unrelated ERC-20 tokens	Low / Info	Design characteristic
AS-05	Pool addresses cannot be changed after deployment	Info	Operational risk
AS-06	Fixed supply / no mint mechanism identified	Info	Positive
AS-07	No proxy / upgrade mechanism identified	Info	Positive
AS-08	Solidity 0.8 arithmetic protections	Info	Positive

*High severity here denotes a material trading/integration configuration risk while the restriction remains enabled; it is not an assertion of a remotely exploitable code vulnerability.

15. Recommended Pre-Launch Checks

• Verify the deployed contract address and that verified source matches deployed bytecode.

• Confirm token name, symbol, decimals, and total supply.

• Confirm the deployer receives the intended initial supply.

• Verify the configured Uniswap V2 and V3 pool addresses.

• Confirm transferConstraints is in the intended state.

• If trading is intended, execute removeTransferConstraints() only after verifying configuration.

• Perform small buy and sell tests and confirm expected DEX behavior.

• Test transfer(), approve(), and transferFrom().

• Confirm the owner address and consider a multisig for administrative control.

• Confirm the metadata URI resolves correctly and contains the intended information.

• Review initial distribution, liquidity ownership, and liquidity-lock arrangements separately.

• Perform independent testing or an external security audit before public launch.

16. Overall Conclusion

No obvious Critical or High-severity code-execution, arbitrary-minting, arithmetic, or unauthorized-access vulnerability was identified from the supplied Solidity source. The primary risks are administrative and operational rather than conventional Solidity exploits.

The most material operational issue is the initial restriction on transfers involving the configured Uniswap V2 and V3 pool addresses. Trading therefore requires the owner to remove the transfer constraints. The owner also has authority to recover ETH and unrelated ERC-20 tokens held by the contract.

The token supply is initialized at deployment and no mint function was identified. No proxy or upgrade mechanism was identified in the supplied source. These conclusions are limited to the code and assumptions described in this document.

17. Audit Limitations and Disclaimer

This report was prepared from the Solidity source code supplied for review. It does not constitute an independent third-party security audit, formal verification, guarantee against vulnerabilities, guarantee against economic or market risks, guarantee regarding Uniswap/wallet/exchange/blockchain integration, or certification by Etherscan or any other platform.

The assessment does not independently verify deployed bytecode, deployment parameters, ownership wallet, liquidity configuration, external contracts, compiler settings, or off-chain infrastructure. A professional independent security audit should be obtained before representing the contract as independently audited.

ASSESSMENT CLASSIFICATION
SELF-ASSESSMENT — NOT AN INDEPENDENT SECURITY AUDIT

Contract Reviewed: USDGGold Version: Solidity 0.8.20 Date: 13 August 2026